

Population Dynamics

Spencer Wang

AICM, Period 7

Princeton International School of Mathematics and Science

1 - Driving Program For all

```
import numpy as np
from scipy.integrate import odeint
import matplotlib.pyplot as plt
from matplotlib.collections import LineCollection
import matplotlib as mpl

mpl.rcParams['figure.dpi'] = 250
mpl.rcParams.update({'font.size': 8})

class Whatever:
    def __init__(self, prey_init, predator_init, a, b, c, d, seed=0):
        self.prey_init = prey_init
        self.predator_init = predator_init
        self.a = a
        self.b = b
        self.c = c
        self.d = d
        self.seed = seed
        np.random.seed(self.seed)

        self.main_colors = {
            'background': '#ffffff',
            'vector_field': '#FF8C00',
            'trajectory': '#800080',
            'init_point': '#FF4500',
            'color_map': 'plasma',
        }

        plt.style.use('default')
        mpl.rcParams.update({
            'axes.facecolor': self.main_colors['background'],
            'axes.edgecolor': 'black',
            'axes.labelcolor': 'black',
            'text.color': 'black',
            'xtick.color': 'black',
            'ytick.color': 'black',
            'figure.facecolor': self.main_colors['background'],
            'grid.color': '#cccccc',
            'grid.linestyle': '--',
            'grid.linewidth': 0.5,
        })

    def lotka_volterra(self, state, t): // This is your function
        prey, predator = state
        return [prey, predator]

    def solve_ode(self, t, initial_conditions=None):
```

```

        if initial_conditions is None:
            initial_conditions = [self.prey_init, self.predator_init]
        solution = odeint(self.lotka_volterra, initial_conditions, t)
        return solution

def generate_uniform_initial_conditions(self, num_points, prey_range, predator_range):
    prey_init = np.random.uniform(prey_range[0], prey_range[1], num_points)
    predator_init = np.random.uniform(predator_range[0], predator_range[1], num_points)
    initial_conditions = np.vstack((prey_init, predator_init)).T
    return initial_conditions

def plot_phase_space(self, t, solutions, ax):
    cmap = self.main_colors['color_map']
    norm = plt.Normalize(t.min(), t.max())

    for solution in solutions:
        x = solution[:, 0]
        y = solution[:, 1]

        points = np.array([x, y]).T.reshape(-1, 1, 2)
        segments = np.concatenate([points[:-1], points[1:]], axis=1)

        lc = LineCollection(segments, cmap=cmap, norm=norm)
        lc.set_array(t)
        lc.set_linewidth(2)
        line = ax.add_collection(lc)

        ax.scatter(x[0], y[0], color=self.main_colors['init_point'], zorder=5)

    all_x = np.concatenate([sol[:, 0] for sol in solutions])
    all_y = np.concatenate([sol[:, 1] for sol in solutions])
    ax.set_xlim(all_x.min() * 0.9, all_x.max() * 1.1)
    ax.set_ylim(all_y.min() * 0.9, all_y.max() * 1.1)

    ax.set_xlabel('Rabbits')
    ax.set_ylabel('Foxes')
    ax.set_title('Phase Space Trajectories')

    cbar = plt.colorbar(line, ax=ax, label='Time', pad=0.02)
    cbar.ax.tick_params(labelsize=6)

def plot_vector_field(self, ax, prey_range, predator_range, num_pts=20):
    X, Y = np.meshgrid(np.linspace(*prey_range, num_pts), np.linspace(*predator_range,
num_pts))
    U = self.a * X - self.b * X * Y
    V = self.d * X * Y - self.c * Y

    magnitude = np.sqrt(U**2 + V**2)
    magnitude[magnitude == 0] = 1
    U_norm = U / magnitude
    V_norm = V / magnitude

    ax.quiver(X, Y, U_norm, V_norm, color=self.main_colors['vector_field'],
        alpha=0.6, pivot='mid', scale=30, width=0.005)

    ax.set_title('Vector Field')

```

```

def plot_derivative_colored(self, t, solutions, ax):
    cmap = self.main_colors['color_map']

    for solution in solutions:
        x = solution[:, 0]
        y = solution[:, 1]

        dx = self.a * x - self.b * x * y
        dy = self.d * x * y - self.c * y
        deriv_magnitude = np.sqrt(dx**2 + dy**2)

        norm = plt.Normalize(deriv_magnitude.min(), deriv_magnitude.max())

        points = np.array([x, y]).T.reshape(-1, 1, 2)
        segments = np.concatenate([points[:-1], points[1:]], axis=1)

        lc = LineCollection(segments, cmap=cmap, norm=norm)
        lc.set_array(deriv_magnitude)
        lc.set_linewidth(2)
        line = ax.add_collection(lc)

        ax.scatter(x[0], y[0], color=self.main_colors['init_point'], zorder=5)

    all_x = np.concatenate([sol[:, 0] for sol in solutions])
    all_y = np.concatenate([sol[:, 1] for sol in solutions])
    ax.set_xlim(all_x.min() * 0.9, all_x.max() * 1.1)
    ax.set_ylim(all_y.min() * 0.9, all_y.max() * 1.1)

    ax.set_xlabel('Rabbits')
    ax.set_ylabel('Foxes')
    ax.set_title('Trajectories Colored by Derivative Magnitude')

    cbar = plt.colorbar(line, ax=ax, label='Derivative Magnitude', pad=0.02)
    cbar.ax.tick_params(labelsize=6)

def plot_time_series(self, t, solutions, ax1, ax2):
    cmap = self.main_colors['color_map']
    colors = plt.cm.plasma(np.linspace(0, 1, len(solutions)))

    for idx, solution in enumerate(solutions):
        x = solution[:, 0]
        y = solution[:, 1]

        dx = self.a * x - self.b * x * y
        dy = self.d * x * y - self.c * y

        ax1.plot(t, x, color=colors[idx], label=f'Rabbits Traj {idx+1}')
        ax1.plot(t, y, '--', color=colors[idx], label=f'Foxes Traj {idx+1}')

        ax2.plot(t, dx, color=colors[idx], label=f'Rabbits' Derivative {idx+1}')
        ax2.plot(t, dy, '--', color=colors[idx], label=f'Foxes' Derivative {idx+1}')

    ax1.set_xlabel('Time')
    ax1.set_ylabel('Population')
    ax1.set_title('Population Over Time')
    ax1.legend(fontsize=6, loc='upper right')

```

```

ax2.set_xlabel('Time')
ax2.set_ylabel('Derivative')
ax2.set_title('Derivative Over Time')
ax2.legend(fontsize=6, loc='upper right')

def plot_all(self, t, solutions):
    fig, axes = plt.subplots(1, 3, figsize=(18, 6), constrained_layout=True)
    traj_ax, vecfield_ax, deriv_color_ax = axes

    self.plot_phase_space(t, solutions, traj_ax)

    all_x = np.concatenate([sol[:, 0] for sol in solutions])
    all_y = np.concatenate([sol[:, 1] for sol in solutions])
    prey_min, prey_max = all_x.min() * 0.9, all_x.max() * 1.1
    predator_min, predator_max = all_y.min() * 0.9, all_y.max() * 1.1

    self.plot_vector_field(vecfield_ax, [prey_min, prey_max], [predator_min,
predator_max])

    self.plot_derivative_colored(t, solutions, deriv_color_ax)

    plt.show()

    fig_ts, axes_ts = plt.subplots(1, 2, figsize=(12, 5), constrained_layout=True)
    ax1, ax2 = axes_ts
    self.plot_time_series(t, solutions, ax1, ax2)
    plt.show()

def run_simulation(self, max_t=15, num_points=1000, num_initial_conditions=1,
    prey_range=(1, 20), predator_range=(1, 20)):
    t = np.linspace(0, max_t, num_points)

    initial_conditions =
self.generate_uniform_initial_conditions(num_initial_conditions,
    prey_range, predator_range)

    solutions = []
    for init_cond in initial_conditions:
        solution = self.solve_ode(t, initial_conditions=init_cond)
        solutions.append(solution)

    self.plot_all(t, solutions)

```

2 - Exercise 1

2.1 - Driving Function

```

def lotka_volterra(self, state, t):
    prey, predator = state
    dprey_dt = self.a * prey - self.b * prey * predator
    dpredator_dt = self.d * prey * predator - self.c * predator
    return [dprey_dt, dpredator_dt]

```

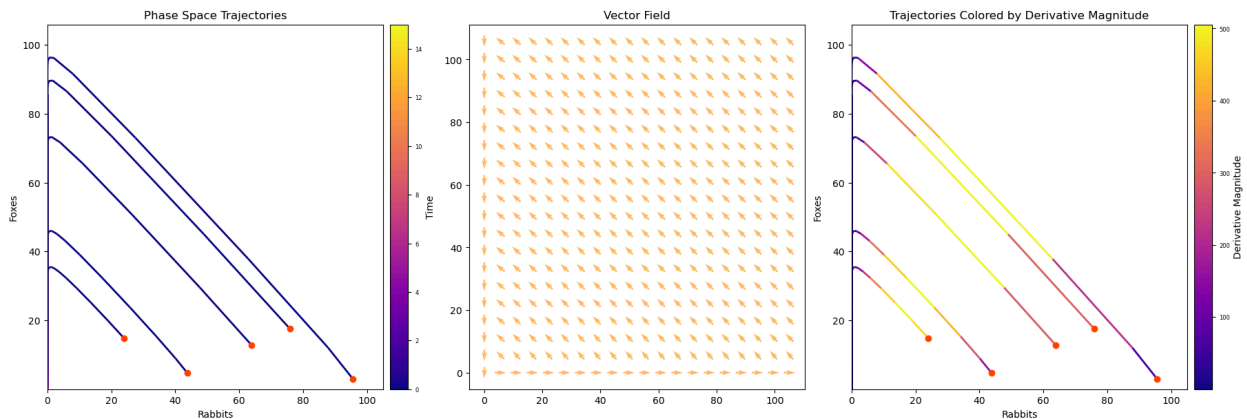
2.2 - Analysis

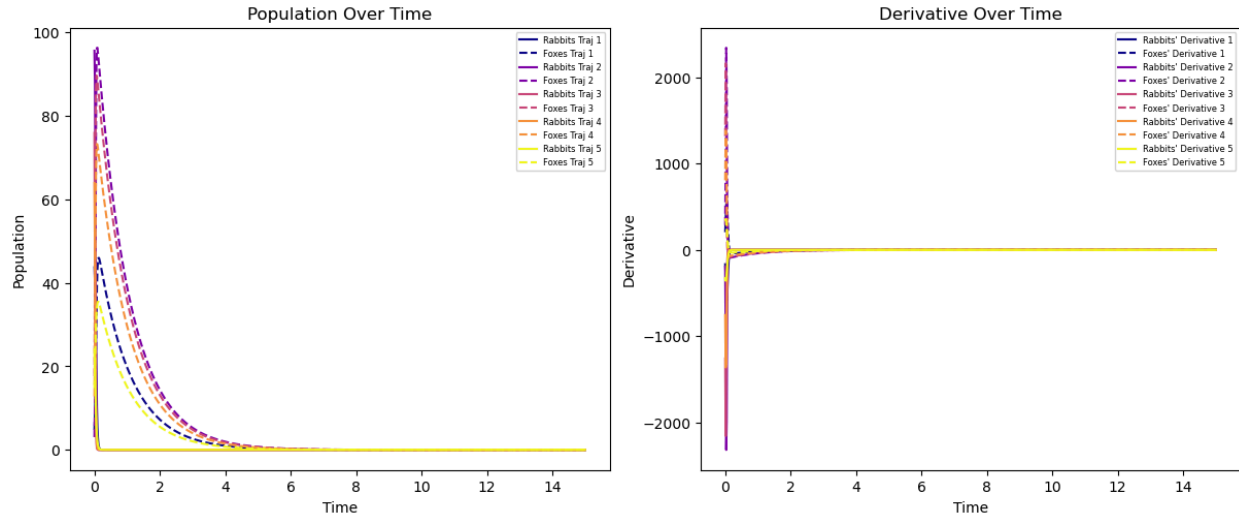
We can then analyze what happens when all parameters a, b, c, d are the same. Meaning that the predation of fox on rabbit, rabbit's reproduction rate, and the competition between foxes have the same strength (although doesn't make sense irl).

```
prey_initial = 10
predator_initial = 5
a = 1.0
b = 1.0
c = 1.0
d = 1.0

rf = RabbitFoxes(pre_init=prey_initial, predator_init=predator_initial,
                 a=a, b=b, c=c, d=d, seed=42)

rf.run_simulation(
    max_t=15,
    num_points=1000,
    num_initial_conditions=5,
    prey_range=(10, 100),
    predator_range=(2, 20)
)
```





Here if we plot properly, we can see that the system will get absolutely destroyed. The rabbit population will go to 0, and then the fox population will also go to 0. Thus, ggs. Notice that the initial condition of the rabbit-fox system do indeed matter (a bit). Meaning, two different initial values will give us two different cycling patterns.

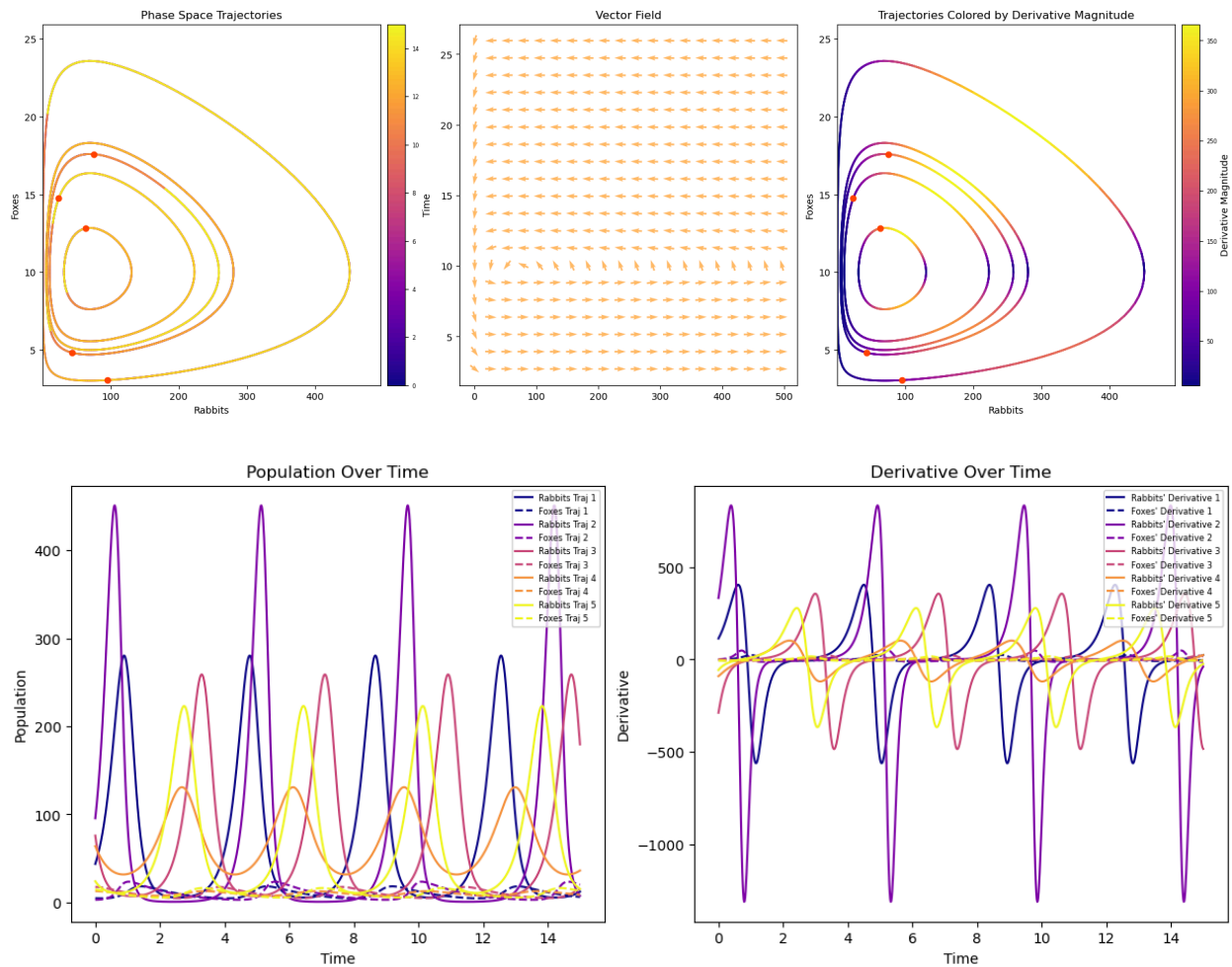
```

prey_initial = 10
predator_initial = 5
a = 5.0
b = 0.5
c = 0.7
d = 0.01

rf = RabbitFoxes(prey_init=prey_initial, predator_init=predator_initial,
                  a=a, b=b, c=c, d=d, seed=42)

rf.run_simulation(
    max_t=15,
    num_points=1000,
    num_initial_conditions=5,
    prey_range=(10, 100),
    predator_range=(2, 20)
)

```



Here parameters are what they are. The meaning is rather intuitive so yeah. Here, this is a nice example of a cycling system. There is no point of “equilibrium” in this system. The rabbit and fox populations will just keep cycling.

However, the initial conditions do matter. If we change the initial conditions, the cycling pattern will change. And actually, will decide whether the system will go to extinction or not (not mathematically, but to our human eyes).

2.3 - Unrealistic

This system is pretty unrealistic if we think about it. Like how are the rabbit population just unbounded? Also, I don’t believe the interaction term can be simply treated as a linear term. But it’s a nice example of a simple system that can be solved analytically and demonstrated clearly with a graph.

Like look at this...

```

prey_initial = 10
predator_initial = 5
a = 0.2
b = 1.0
c = 1.0
d = 1.0

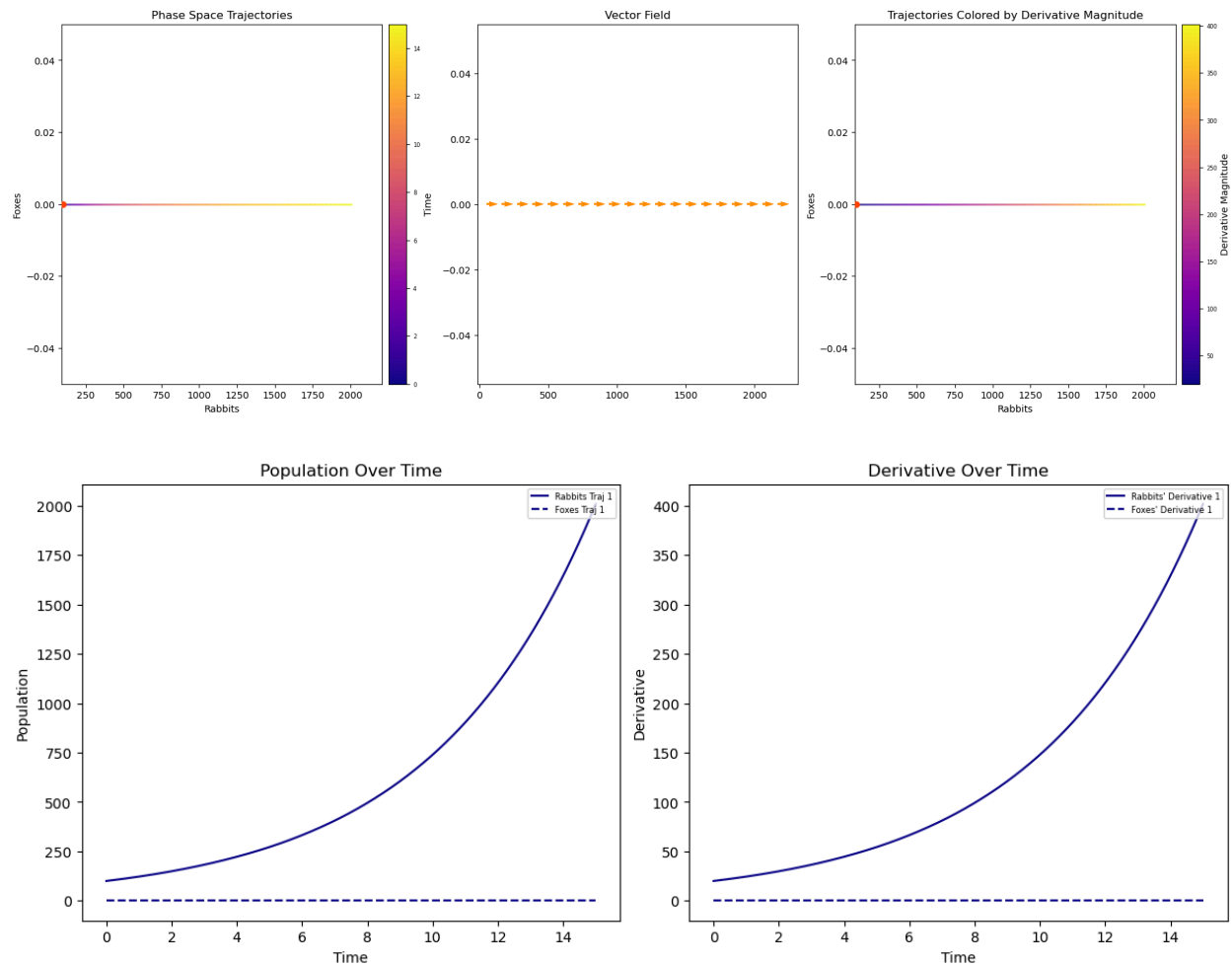
```

```

rf = RabbitFoxes(pre_y_init=prey_initial, predator_init=predator_initial,
                 a=a, b=b, c=c, d=d, seed=42)

rf.run_simulation(
    max_t=15,
    num_points=1000,
    num_initial_conditions=1,
    prey_range=(100, 100),
    predator_range=(0, 0)
)

```



2.4 - Fix

So we can just make the first term of the rabbit bounded by a logistic growth term. This is a simple fix to the problem. Leading to the following function:

```

def lotka_volterra(self, state, t):
    prey, predator = state
    dprey_dt = self.a * prey * (1 - prey / self.K) - self.b * prey * predator
    dpredator_dt = self.d * prey * predator - self.c * predator
    return [dprey_dt, dpredator_dt]

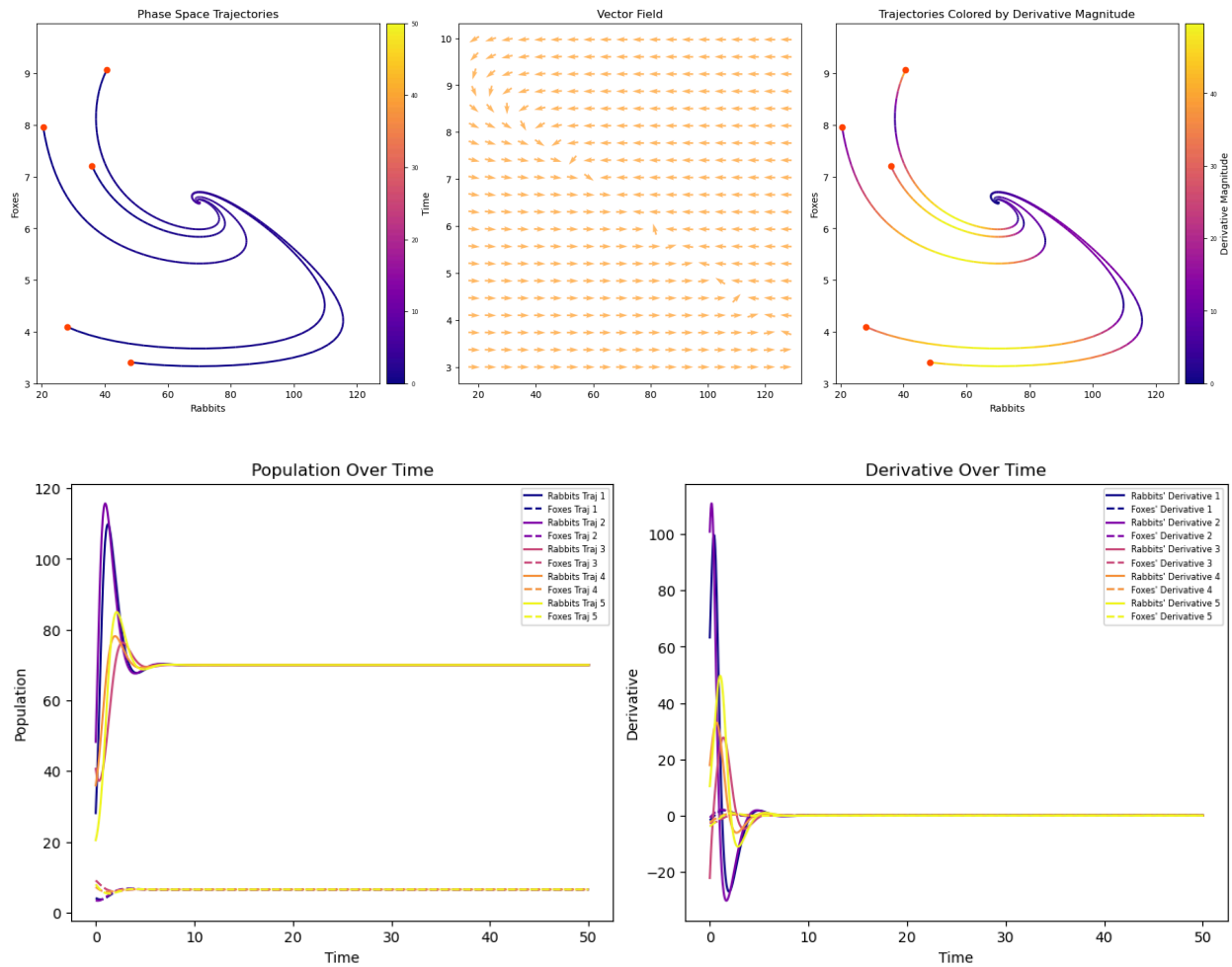
```

This function introduced a new variable: K , the carrying capacity of rabbits.

```
a = 5.0
b = 0.5
c = 0.7
d = 0.01
K = 200

rf = RabbitFoxes(a=a, b=b, c=c, d=d, K=K, seed=42)

rf.run_simulation(
    max_t=50,
    num_points=1000,
    num_initial_conditions=5,
    prey_range=(15, 50),
    predator_range=(3, 10)
)
```



Now we can observe that they all enter a fixed point, which is good (now the whole thing is stable). The rabbit population is bounded by the carrying capacity, and the fox population is bounded by the rabbit

population. This is a more realistic model of the system. This basically stops the previously observed trend where the rabbit will increase without bounds and then foxes just start to massacre them. Then the number of foxes got way too out of hand and then started to kill off too many rabbits, leading to an unstable system.

3 - Exercise 2

3.1 - Driving Program

Basically the same last, just with a slight modified function and initial conditions.

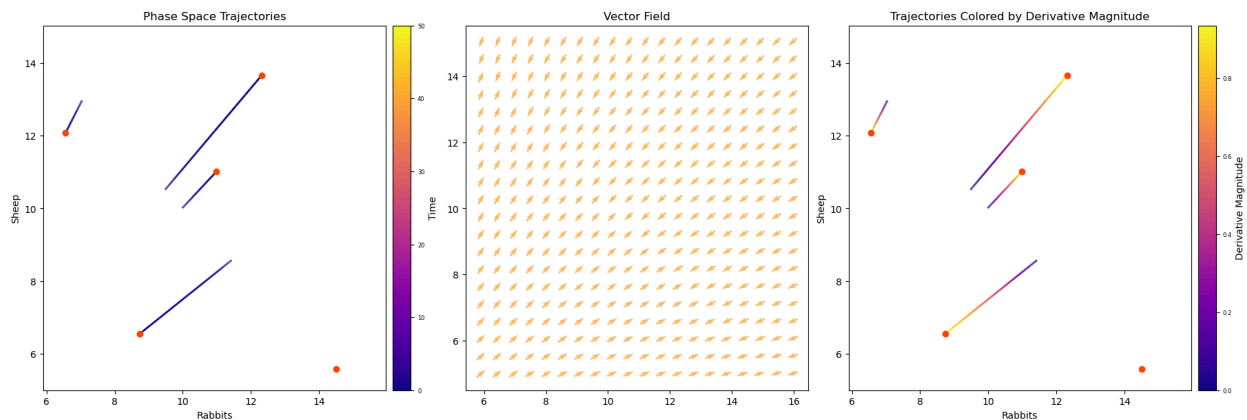
```
def competition_model(self, state, t):
    prey1, prey2 = state
    dprey1_dt = self.ra * (1 - prey1 / self.ka) * prey1 - self.a * prey1 * prey2
    dprey2_dt = self.rb * (1 - prey2 / self.kb) * prey2 - self.b * prey1 * prey2
    return [dprey1_dt, dprey2_dt]
```

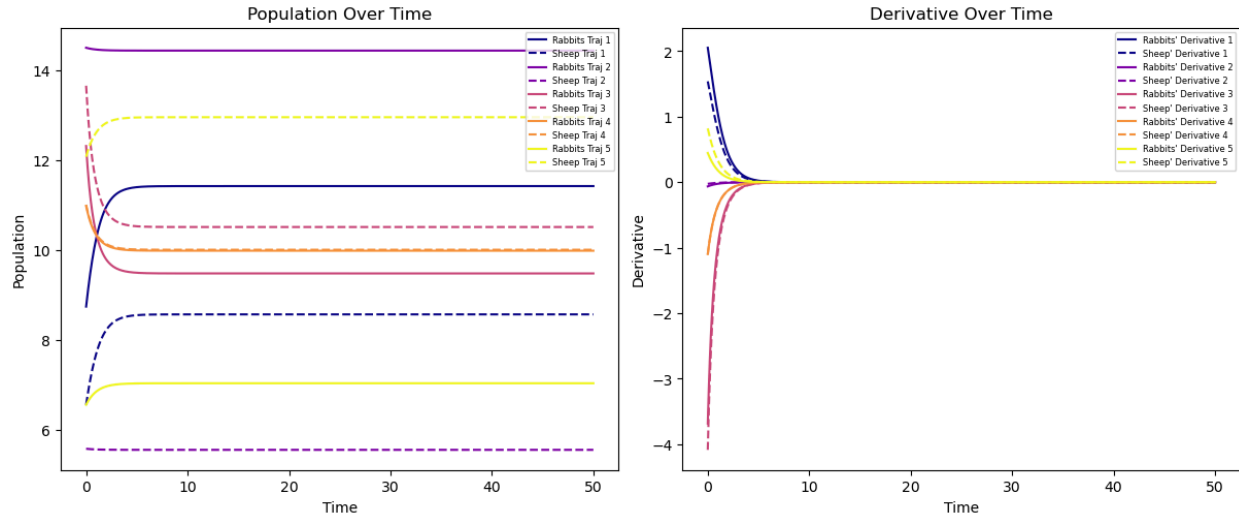
3.2 - Analysis

```
a = 0.05
b = 0.05
ra = 1.0
rb = 1.0
ka = 20
kb = 20

rs = RabbitSheep(a=a, b=b, ra=ra, rb=rb, ka=ka, kb=kb, seed=42)

rs.run_simulation(
    max_t=50,
    num_points=1000,
    num_initial_conditions=5,
    prey1_range=(5, 15),
    prey2_range=(5, 15)
)
```



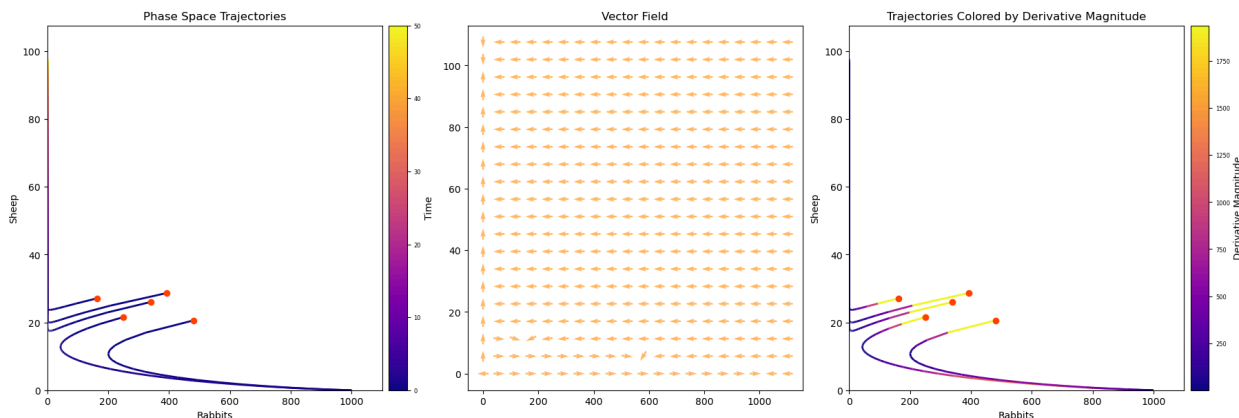


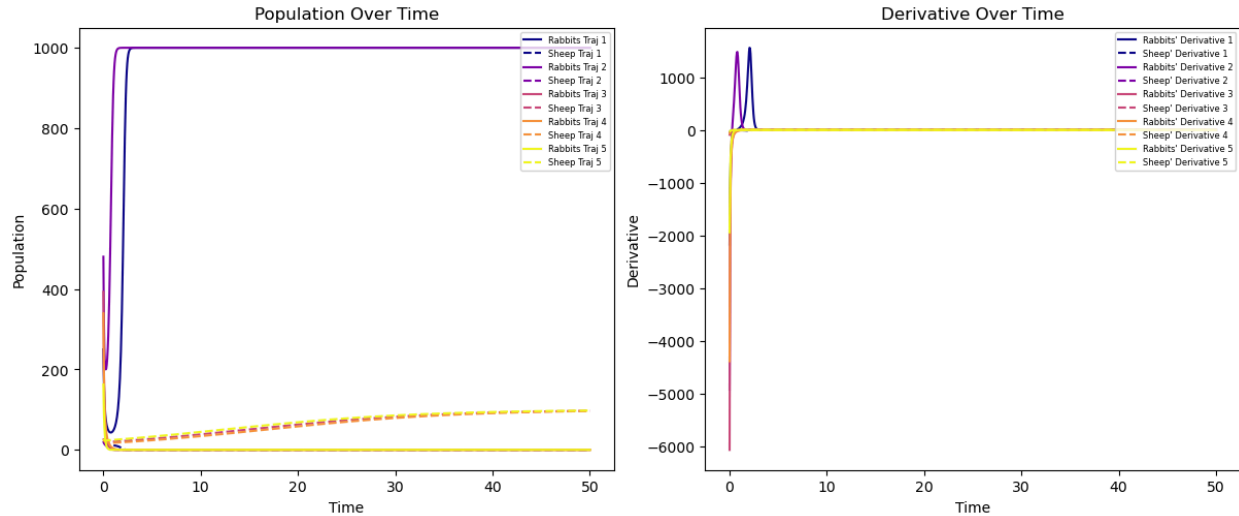
Here are some unrealistic parameters. Here we observe that they all reach an equilibrium state where both numbers are equal. This is really really easy to understand why, and yeah. However, the parameters are pretty unrealistic because rabbits are not really as competitive as sheets and the growth rate of rabbit should be much higher than sheep.

```
a = 0.75
b = 0.01
ra = 10.0
rb = 0.1
ka = 1000
kb = 100

rs = RabbitSheep(a=a, b=b, ra=ra, rb=rb, ka=ka, kb=kb, seed=42)

rs.run_simulation(
    max_t=50,
    num_points=1000,
    num_initial_conditions=5,
    prey1_range=(100, 500),
    prey2_range=(20, 30)
)
```





With this setup, we can nicely see the two extremities of initial conditions, where some setup can make rabbits go extinct or sheep go extinct. This is a nice example of how the initial conditions can affect the system.

However, we still can't see a state where sheep and rabbit had a chance to stay relatively stable. This could be because my setups are still a bit off.

Despite my attempts, I wasn't able to make a set of parameters where both will reach a stable state.

3.3 - Unrealistic

This model is pretty realistic overall, and I couldn't see anything that I can improve on. Perhaps one scenerio where I can do is that setting by modifying how sheep and rabbits fight for the same food. For example, like rabbits are more likely to content for food because there are larger populations of them. However, this can also be reflected through tuning parameters.

3.4 - Fix

So I've just modified how competitive rabbits are to sheep. Because previously, we are all thinking about how rabbits are weaker than sheep and would just run away. However, we've neglected the fact that sheep would also have less population. So basically, they will have less area of grass to eat. Thus, making rabbits somewhat competitive. Also, sheep don't die to rabbits, so we can also set that to be 0.

```

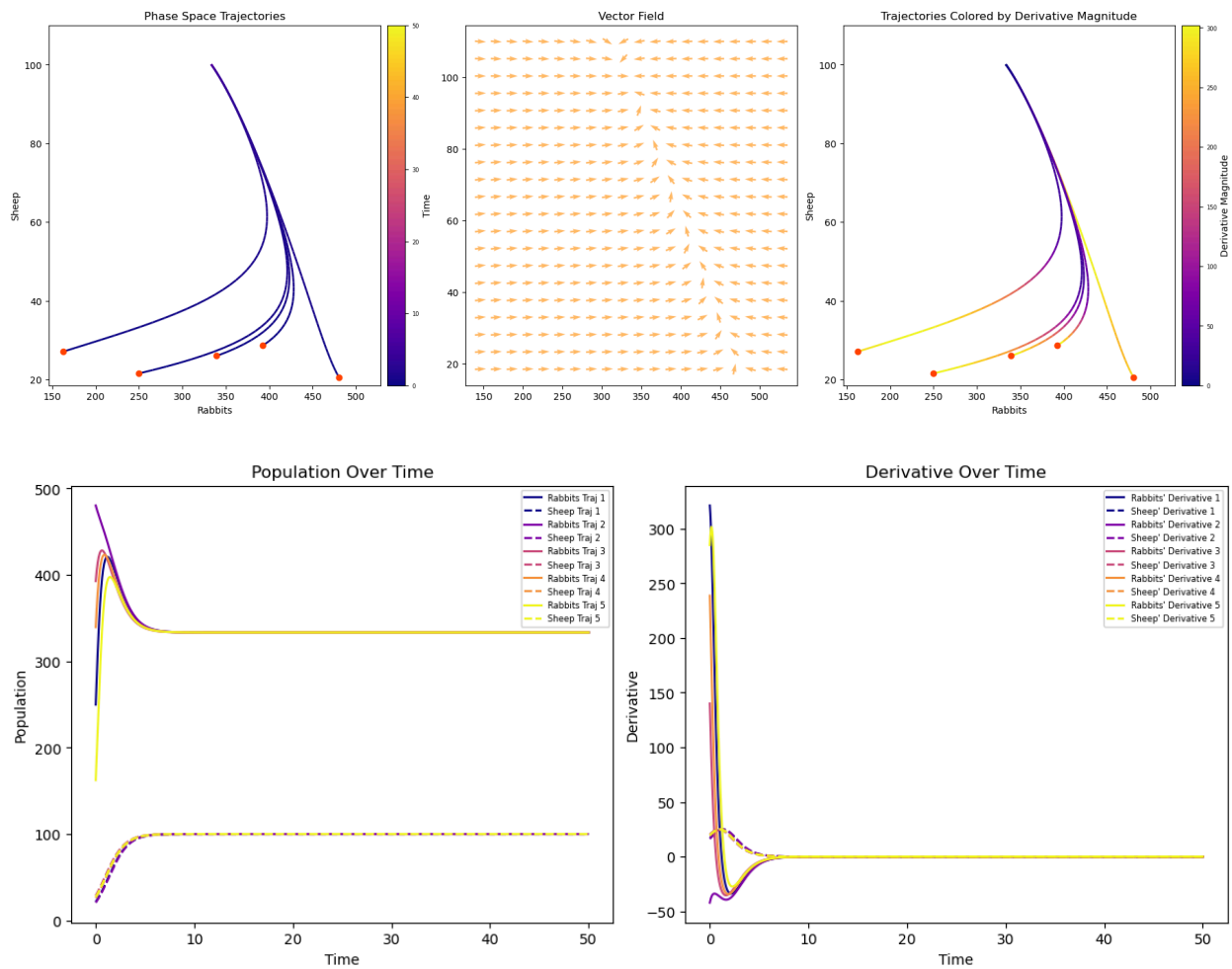
a = 0.01
b = 0
ra = 3.0
rb = 1
ka = 500
kb = 100

rs = RabbitSheep(a=a, b=b, ra=ra, rb=rb, ka=ka, kb=kb, seed=42)

rs.run_simulation(
    max_t=50,
    num_points=1000,
    num_initial_conditions=5,
    preyl_range=(100, 500),

```

```
)    prey2_range=(20, 30)
```



Well, now they converge into a steady state. Where the whole thing reaches an equilibrium state.

4 - Exercise 3

4.1 - Math Stuff

We have:

$$\begin{cases} \frac{dR}{dt} = r_R \left(1 - \frac{R}{K_R}\right) R - a \cdot f(S) \cdot R \cdot S \\ \frac{dS}{dt} = r_S \left(1 - \frac{S}{K_S}\right) S - b \cdot f(S) \cdot R \cdot S \end{cases}$$

Where we define F as:

$$f(S) = \begin{cases} 0 & \text{if } S \leq F_{p_{\max}} \\ 1 - \frac{F_{p_{\max}}}{S} & \text{if } S > F_{p_{\max}} \end{cases}$$

We can then fully expand out the whole thing to be like:

$$\begin{aligned} \frac{dR}{dt} &= r_R \left(1 - \frac{R}{K_R}\right) R - \begin{cases} 0 & \text{if } S \leq F_{p_{\max}} \\ a \left(1 - \frac{F_{p_{\max}}}{S}\right) R \cdot S & \text{if } S > F_{p_{\max}} \end{cases} \\ \frac{dS}{dt} &= r_S \left(1 - \frac{S}{K_S}\right) S - \begin{cases} 0 & \text{if } S \leq F_{p_{\max}} \\ b \left(1 - \frac{F_{p_{\max}}}{S}\right) R \cdot S & \text{if } S > F_{p_{\max}} \end{cases} \end{aligned}$$

(I don't know how to do the brackets in typst, so I did a screenshot instead)

Reasoning is below.

4.2 - Reasoning and Driving Code

Well now, since the sheep have an area where they can eat food and rabbits don't have access to that field. Like, whenever sheep run out of food in common area, they just go there... and rabbits can't do anything about it. The competitiveness of sheep to rabbits should also decrease as sheep will also tend to go to the private area.

Then in this case, we have to add an area where there is a specific amount of food that is only accessible to sheep. If the maximum is reached, sheep will still have to fight the rabbits for food.

Thus, we can arrive at the following:

```
def competition_model(self, state, t):
    prey1, prey2 = state
    if prey2 > self.F_p_max:
        f = 1 - self.F_p_max / prey2
    else:
        f = 0
    dprey1_dt = self.ra * (1 - prey1 / self.ka) * prey1 - self.a * f * prey1 * prey2
    dprey2_dt = self.rb * (1 - prey2 / self.kb) * prey2 - self.b * f * prey1 * prey2
    return [dprey1_dt, dprey2_dt]
```

With the following setup:

```
a = 0.01
b = 0.01
ra = 1.0
rb = 0.8
ka = 100
kb = 80
F_p_max = 30
seed = 42

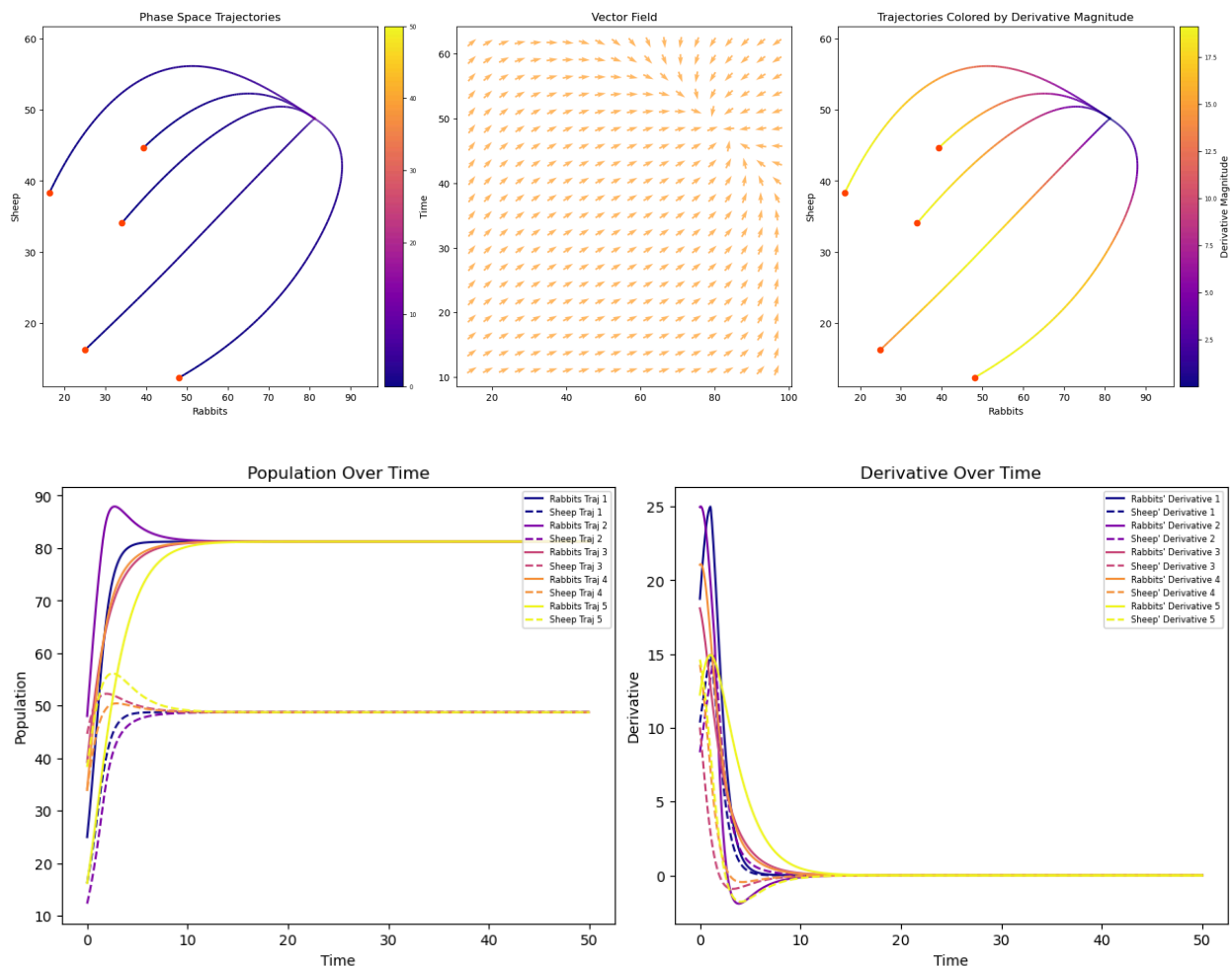
simulation = RabbitSheep(a, b, ra, rb, ka, kb, F_p_max, seed)

simulation.run_simulation(
    max_t=50,
```

```

num_points=1000,
num_initial_conditions=5,
prey1_range=(10, 50),
prey2_range=(10, 50)
)

```



We can observe that the whole thing reaches a stable point again. This is a nice example of how we can modify the system to make it more realistic.

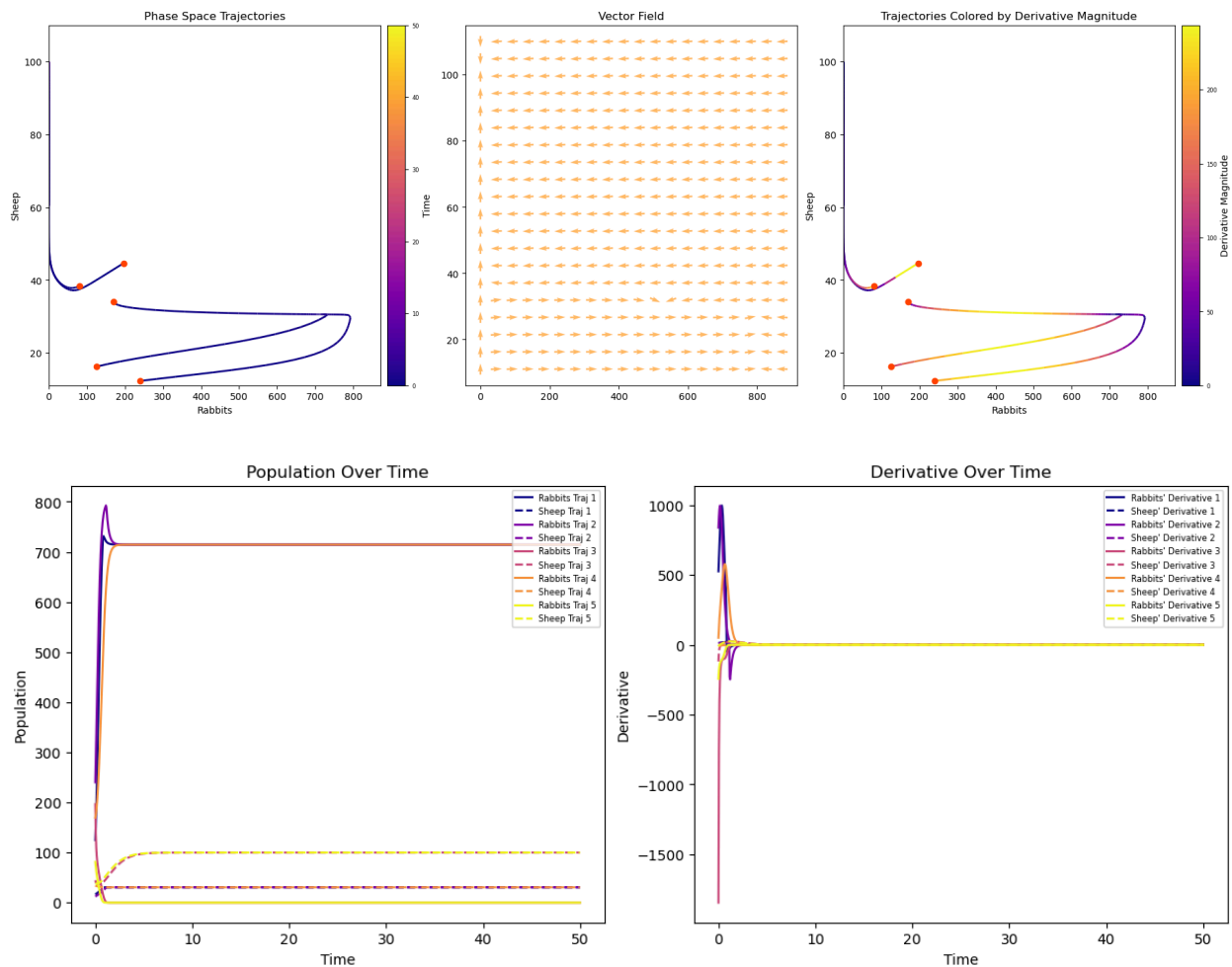
```

a = 0.9      # Competition rate coefficient from rabbits to sheep
b = 0.05     # Competition rate coefficient from sheep to rabbits
ra = 5.0     # Intrinsic growth rate of rabbits
rb = 1       # Intrinsic growth rate of sheep
ka = 800     # Carrying capacity for rabbits
kb = 100     # Carrying capacity for sheep
F_p_max = 30 # Maximum food capacity in the private area for sheep
seed = 42    # Seed for reproducibility

# Instantiate the simulation with the private food area parameter
simulation = RabbitSheep(a, b, ra, rb, ka, kb, F_p_max, seed)

```

```
# Run the simulation
simulation.run_simulation(
    max_t=50,
    num_points=1000,
    num_initial_conditions=5,
    prey1_range=(50, 250),
    prey2_range=(10, 50) # Extended range to observe behavior when prey2 > F_p_max
)
```



With the set of more “reallistic” values from exercise 2, we can observe the above results. We see that with different starting conditions, results can converge differently. Where some initial conditions favors one animal to dominate over the other, while other initial conditions tends to lead to a stable environment. With the vector field given, we can observe fixed points where equilibrium will occur.

5 - Exercise 4

Here, it’s just really a matter of solving the thing. We are given what seeming like little to work with. However, we have just enough information to piece everything together to form the final solution.

$$\dot{R} = (r_R - \frac{r_R}{k_R} R - bS) R$$

$$-1.2 =$$

$$\dot{S} = (r_S - \frac{r_S}{k_S} S - dR) S$$

$$-0.133 =$$

$$(r_R - \frac{r_R}{k_R} R - bS) R = 0$$

$$r_S - \frac{r_S}{k_S} S - dR = 0.3$$

$$(r_S - \frac{r_S}{k_S} S - dR) S = 0$$

$$\hookrightarrow r_S = 0.3$$

$$\begin{cases} r_R - \frac{r_R}{k_R} (4.44) - b(0.65) = 0 \\ r_S - \frac{r_S}{k_S} (0.65) - d(4.44) = 0 \end{cases}$$

$$k_R = 6$$

$$r_S = 0.3$$

$$\begin{cases} r_R - \frac{r_R}{6} (4.44) - b(0.65) = 0 \\ 0.3 - \frac{0.3}{k_S} (0.65) - d(4.44) = 0 \end{cases} \quad \begin{cases} (r_R - \frac{r_R}{6} (6) - b(1))6 = -1.2 \\ (0.3 - \frac{0.3}{k_S} (1) - d(6))1 = -0.133 \end{cases}$$

$$r_R = 0.5, b = 0.2$$

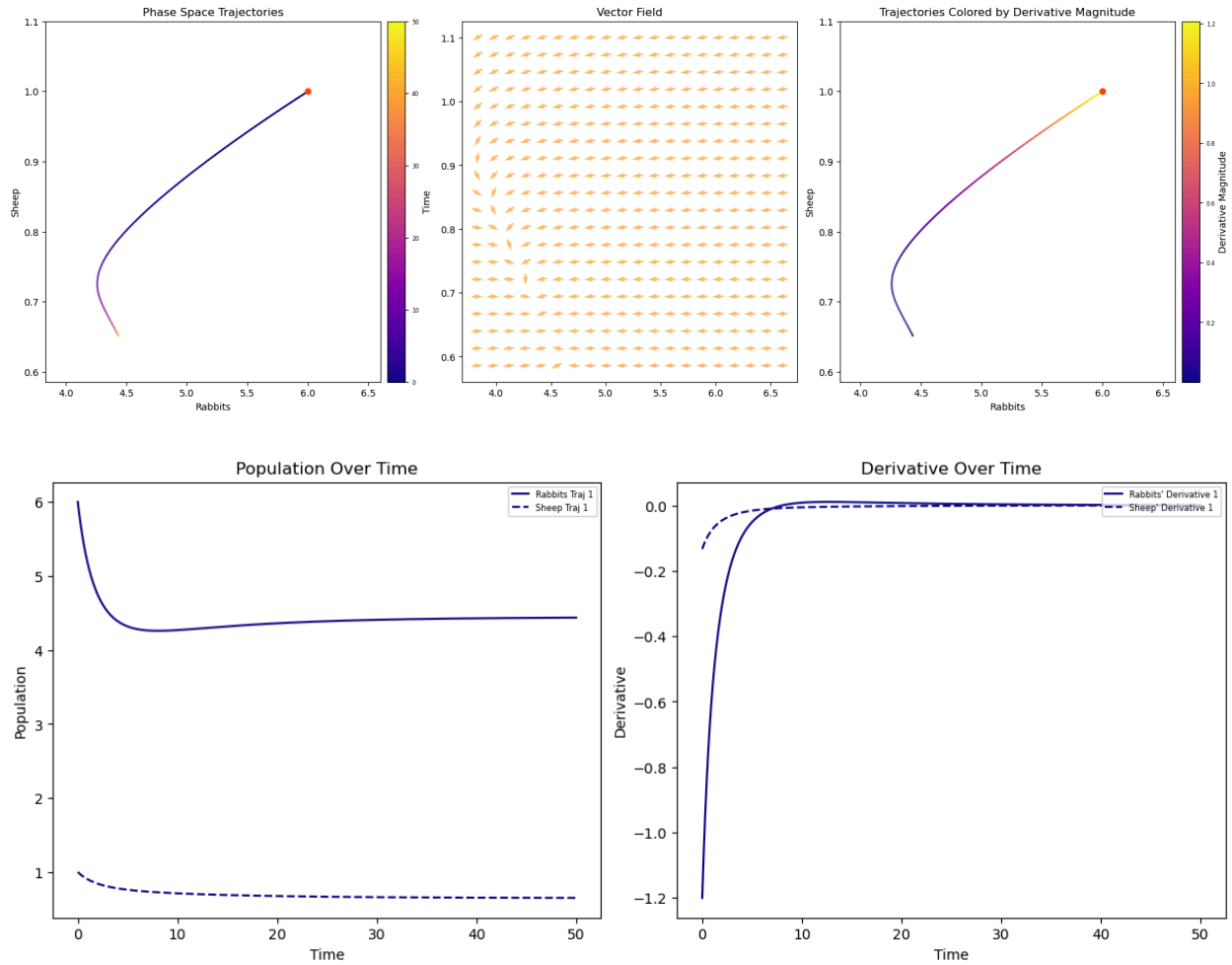
$$k_S = 1.32, d = 0.0343519$$

So now, we can plug in these values to test out the system.

```
a = 0.2
b = 0.0343519
ra = 0.5
rb = 0.3
ka = 6
kb = 1.32
```

```
rs = RabbitSheep(a=a, b=b, ra=ra, rb=rb, ka=ka, kb=kb, seed=42)
```

```
rs.run_simulation(
    max_t=50,
    num_points=1000,
    num_initial_conditions=1,
    prey1_range=(6, 6),
    prey2_range=(1, 1)
)
```



So now, we can see that this system converges to a stable population state where both rabbit and sheeps have a stable population. Nice. :)

6 - Extra Exercise

6.1 - Math Stuff

We can follow the following steps to non-dimensionalize the system:

$$\dot{R} = aR - bRF$$

$$\dot{F} = -cF + dRF$$

$$R = \alpha r \quad dR = \alpha dr$$

$$F = \beta f \quad dF = \beta df$$

$$T = \phi t \quad dT = \phi dt$$

$$\left\{ \begin{array}{l} \frac{\alpha dr}{\phi dt} = a \cdot \alpha r - b(\alpha r)(\beta f) \\ \frac{\beta df}{\phi dt} = -c(\beta f) + d(\alpha r)(\beta f) \end{array} \right.$$

$$\left\{ \begin{array}{l} \frac{dr}{dt} = (a\alpha)r - (b\alpha\beta)rf \\ \frac{df}{dt} = (-c\beta)f + (d\alpha\beta)rf \end{array} \right. \frac{\phi}{\alpha}$$

$$\left\{ \begin{array}{l} \frac{dr}{dt} = (a\alpha)r - (b\alpha\beta)rf \\ \frac{df}{dt} = (-c\beta)f + (d\alpha\beta)rf \end{array} \right. \frac{\phi}{\beta}$$

$$\phi a \quad \textcircled{1} \quad \phi = \frac{1}{a}$$

$$\phi \beta b \quad \textcircled{1} \quad \beta = \frac{a}{b}$$

$$\phi c \quad \left(\frac{c}{a} \right)$$

$$\phi \alpha d \quad \textcircled{1} \quad \alpha = \frac{a}{d}$$

$$\boxed{\begin{cases} \dot{r} = r - rf \\ \dot{f} = -\frac{c}{a}f + rf \end{cases}}$$

$$\frac{dR}{dt} = r_R \left(1 - \frac{R}{k_R}\right) R - aRS$$

$$\frac{ds}{dt} = r_S \left(1 - \frac{S}{k_S}\right) S - bRS$$

$$\begin{cases} R = \alpha r \\ S = \beta s \\ T = \phi t \end{cases} \Rightarrow \begin{cases} dR = \alpha dr \\ ds = \beta ds \\ dT = \phi dt \end{cases}$$

$$\begin{cases} \frac{dr}{dt} = \left(r_R \alpha r - \frac{r_R \cdot \alpha^2 r^2}{k_R} - a \cdot \alpha \cdot \beta (rs) \right) \frac{\phi}{\alpha} \\ \frac{ds}{dt} = \left(r_S \beta s - \frac{r_S \beta^2 s^2}{k_S} - b \cdot \alpha \cdot \beta (rs) \right) \frac{\phi}{\beta} \end{cases}$$

$$\left(\begin{array}{l} \frac{dr}{dt} = r - r^2 - rs \\ \frac{ds}{dt} = \left(\frac{r_S}{r_R} \right) s - \left(\frac{r_S}{k_S} \right) s^2 - \left(\frac{b k_R}{r_R} \right) rs \end{array} \right.$$

$r_R \phi$	①	$\phi = \frac{1}{r_R}$
$r_R \alpha \phi / k_R$	①	$\alpha = k_R$
$a \beta \phi$	①	$\beta = \frac{r_R}{k_R}$
$r_S \phi$	$\frac{r_S}{r_R}$	
$r_S \beta \phi / k_S$	$\frac{r_S}{k_S}$	
$b \cdot \alpha \phi$	$\frac{b k_R}{r_R}$	

$$\frac{ds}{dt} = \left(\frac{r_S}{r_R} \right) s - \left(\frac{r_S}{k_S} \right) s^2 - \left(\frac{b k_R}{r_R} \right) rs$$

Ofcourse, there are more than one way to non-dimensionalize this system. With my initial setup and choice of reduction towards the end, I ended up with these.